

Covariance Estimation, Line by Line

An implementation-level manual for GradientFlowRGToolkit

GradientFlowRGToolkit

31 August 2026

Contents

1	What this document explains	2
2	The common statistical target	3
2.1	Input shape and notation	3
2.2	One joint estimate preserves cross-channel covariance	3
3	Shared estimator contract and evidence	4
3.1	A narrow interchangeable interface	4
3.2	Common validation	5
3.3	Returned scientific value	5
4	Covariance factors and exact correlation propagation	5
4.1	Why several estimators construct a factor	5
4.2	Dense and latent <code>gvar</code> paths	6
4.3	Numerical rank evidence	6
5	Bartlett/Newey–West estimator	7
5.1	Public configuration	7
5.2	Mathematical estimator	7
5.3	Moving sums produce the Bartlett kernel exactly	7
5.4	Exact correction for sample centering under IID data	8
5.5	Structural PSD	9
5.6	Bandwidth-stability diagnostic	9
5.7	What the stability diagnostic does not prove	10
6	Multivariate non-overlapping batch means	10
6.1	Construction	10
6.2	Code mapping	11
6.3	A complete scalar worked example	11
6.4	Tail-discard semantics	12
6.5	Statistical limits	12
7	Positive-leading-bias over-lugsail batch means	12
7.1	Why this method exists	12
7.2	Configuration invariants	12
7.3	Two ordinary batch estimates on one used sample	13
7.4	Why projection can be necessary	13
7.5	Recommended interpretation	14
8	Projected scalar Wolff validation	14
8.1	Its role is validation, not matrix assembly	14

8.2	Lag covariance and accumulated variance	15
8.3	Integrated and effective exponential times	15
8.4	Automatic window selection	16
8.5	Constant and unresolved coordinates	17
9	Experimental rectangular lag-sum heuristic	17
9.1	Scientific status	17
9.2	Diagonal window probe	17
9.3	One common rectangular matrix window	18
9.4	Appropriate use	18
10	Covariance projection evidence	19
11	Reading the result report	19
12	Method-by-method comparison	20
13	Failure modes and what is deliberately not promised	20
13.1	No distribution-free finite-data upper bound	20
13.2	Rank deficiency is not repaired	20
13.3	Full covariance does not mean every direction is diagnosed	21
14	Validation oracles in the test suite	21
14.1	Perfect cross-channel correlation	21
14.2	Exact IID zero-lag correction	21
14.3	Independent batch-means arithmetic	21
14.4	Known VAR(1) long-run covariance	21
14.5	Positive-correlation lugsail behavior	21
14.6	Unresolved-but-available and strict rejection	22
15	Practical configuration patterns	22
15.1	Exploratory primary analysis	22
15.2	Strict publication gate	22
15.3	Independent batch-size sweep	22
15.4	Positive-leading-bias variation	22
16	Computational characteristics	23
17	Source ledger	23
18	File map	23

1 What this document explains

This is a code-oriented companion to the shorter *Long-Run Covariance Methods* guide. It documents every covariance estimation technique currently implemented in `src/gfrgtoolkit/stages/statistics`, including the shared machinery that makes the estimators useful to the rest of the toolkit.

The intended reader should be able to answer all of the following after reading it:

1. What matrix is being estimated?
2. Which array axis represents configurations and which represents measured values?
3. Where do cross-observable and cross-flow-time covariances enter?
4. Why is each normalization factor present?
5. Which estimators are positive semidefinite by construction?
6. Which estimators can require an eigenvalue projection?
7. What is a literature-defined method, what is toolkit policy, and what is an experimental legacy heuristic?
8. What evidence is retained when the autocorrelation tail cannot be resolved?

The code excerpts are taken from the implementation as it stands on 31 August 2026, with long lines wrapped for print where necessary. They are deliberately small enough to read beside the corresponding formula. The source files remain authoritative if the code and this document ever diverge.

2 The common statistical target

2.1 Input shape and notation

Every estimator accepts one finite, two-dimensional NumPy array

$$Y = (Y_{ti}) \in \mathbb{R}^{N \times p}.$$

The indices have fixed meanings:

- $t = 0, \dots, N - 1$ indexes aligned Monte Carlo configurations;
- $i = 0, \dots, p - 1$ indexes measured values;
- one value coordinate can mean a particular observable at a particular flow time;
- different observable channels measured on the same configuration are different columns of the same matrix.

Let

$$\bar{Y} = \frac{1}{N} \sum_{t=0}^{N-1} Y_t, \quad Z_t = Y_t - \bar{Y}.$$

For a stationary vector process, define the lag-(k) covariance

$$\Gamma(k) = \text{Cov}(Y_t, Y_{t+k}), \quad \Gamma(-k) = \Gamma(k)^\top.$$

The long-run covariance is

$$\Sigma = \sum_{k=-\infty}^{\infty} \Gamma(k).$$

Under a multivariate Markov-chain central limit theorem,

$$\sqrt{N}(\bar{Y} - \mu) \implies \mathcal{N}_p(0, \Sigma), \quad \text{Cov}(\bar{Y}) \simeq \frac{\Sigma}{N}.$$

The implementation returns an estimate of $\text{Cov}(\bar{Y})$, not Σ itself. This convention accounts for the N^2 in the Bartlett implementation and the $a(a - 1)$ in the batch-means implementation.

Wolff introduces the full covariance matrix and the covariance of derived means in Eqs. (2), (4), and (12)–(13) of [Wolff \(2004\)](#). The multivariate spectral-variance literature uses the same target; see [Vats, Flegal, and Jones \(2018\)](#).

2.2 One joint estimate preserves cross-channel covariance

The processing stage does **not** estimate each observable or flow time separately. First, every selected observable history has shape $N \times T$, where T is the number of selected flow times. The arrays are concatenated along their value axis:

```
observable_count = len(configuration.dataset.observables)
time_count = len(selected_times)
joint_histories = np.concatenate([
```

```

        histories[observable]
        for observable in configuration.dataset.observables
    ],
    axis=1,
)
joint_estimate = configuration.averaging.method.estimate(
    joint_histories
)
joint_energy = joint_estimate.values.reshape(
    observable_count,
    time_count,
)

```

For two observables and T flow times, `joint_histories` has shape $N \times 2T$. Its covariance therefore contains four $T \times T$ blocks:

$$\text{Cov}(\bar{Y}) = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

The off-diagonal blocks C_{12} and C_{21} are calculated from paired configuration histories. They are not inferred from scalar autocorrelation times and are not appended after averaging. The later `reshape` changes the view of the correlated values, not their covariance.

Linear observable combinations are also formed at the history level, before the joint estimate:

```

for target, sources in configuration.dataset.combine.items():
    combined = np.zeros_like(next(iter(histories.values())))
    for source, weight in sources.items():
        combined = combined + weight * histories[source]
    histories[target] = combined

```

This ordering enforces the project invariant that observable combinations occur before information is discarded.

3 Shared estimator contract and evidence

3.1 A narrow interchangeable interface

`LongRunCovarianceMethod` is a runtime-checkable protocol. A method owns its configuration and exposes one operation:

```

@runtime_checkable
class LongRunCovarianceMethod(Protocol):
    def estimate(
        self,
        histories: np.ndarray,
    ) -> LongRunCovarianceEstimate:
        """Estimate correlated means from aligned configuration histories."""

```

`setup.py` depends only on this aggregate statistical interface:

```

@dataclass(frozen=True)
class StatisticsConfiguration:
    method: LongRunCovarianceMethod

    def __post_init__(self) -> None:
        if not isinstance(self.method, LongRunCovarianceMethod):
            raise ConfigurationError(
                "statistics method must be a supported "

```

```
        "long-run covariance method"
    )
```

The direction of dependency is intentional: individual statistical modules do not import configuration values from `setup.py`.

3.2 Common validation

All estimators pass through `validate_histories`:

```
values = np.asarray(histories, dtype=float)
if values.ndim != 2:
    raise StatisticsError(
        "Monte Carlo histories must have shape "
        "(configurations, values)"
    )
configuration_count, value_count = values.shape
if configuration_count < 2 or value_count < 1:
    raise StatisticsError(
        f"{method_name} requires at least two "
        "configurations and one value"
    )
if not np.all(np.isfinite(values)):
    raise StatisticsError(
        "Monte Carlo histories contain non-finite values"
    )
```

This establishes a minimum common contract. Method-specific constraints—such as a lag below (N), at least two complete batches, or a lugsail-compatible batch size—are checked by the method that owns them.

3.3 Returned scientific value

Every method returns an immutable pair:

```
@dataclass(frozen=True)
class LongRunCovarianceEstimate:
    values: np.ndarray
    evidence: LongRunCovarianceEvidence
```

`values` contains `gvar` objects. Their means are the estimated vector mean, and their shared covariance is the method's estimate of $\text{Cov}(\bar{Y})$. `evidence` describes how that covariance was obtained. It includes, among other fields:

- the method object and stable estimator name;
- the source URL or DOI;
- (N), (p), numerical rank, and rank tolerance;
- lag or batch policy;
- discarded configurations;
- autocorrelation resolution status and diagnostics;
- covariance projection policy and adjustment size;
- any projected Wolff validation result.

The evidence is attached to every channel view, but it describes the single joint ensemble estimate.

4 Covariance factors and exact correlation propagation

4.1 Why several estimators construct a factor

If a covariance can be written

$$C = F^T F, \quad F \in \mathbb{R}^{q \times p},$$

then C is positive semidefinite by construction because, for every $x \in \mathbb{R}^p$,

$$x^T C x = \|F x\|_2^2 \geq 0.$$

Bartlett and ordinary batch means construct (F) directly. They do not form an indefinite matrix and repair it afterward.

4.2 Dense and latent gvar paths

The helper chooses between an ordinary dense covariance and a latent-factor representation:

```
value_count = factor.shape[1]
if value_count <= factor.shape[0]:
    covariance = factor.T @ factor
    return np.asarray(
        gv.gvar(means, covariance, fast=True),
        dtype=object,
    )

latent = np.asarray(
    gv.gvar(
        np.zeros(factor.shape[0]),
        np.ones(factor.shape[0]),
    ),
    dtype=object,
)
return np.asarray(means + factor.T @ latent, dtype=object)
```

In the second path, let $u \in \mathbb{R}^q$ contain independent standard Gaussian `gvar` variables. Then

$$X = \bar{Y} + F^T u$$

has exactly

$$\text{Cov}(X) = F^T \text{Cov}(u) F = F^T F.$$

This avoids forcing a dense $p \times p$ matrix merely to preserve an intrinsically low-rank covariance. It does not diagonalize the covariance or discard off-diagonal information.

4.3 Numerical rank evidence

Rank is calculated from the singular values of (F):

```
singular_values = np.linalg.svd(factor, compute_uv=False)
singular_tolerance = (
    max(factor.shape)
    * np.finfo(float).eps
    * singular_values[0]
)
numerical_rank = int(
    np.count_nonzero(singular_values > singular_tolerance)
)
rank_tolerance = singular_tolerance * singular_tolerance
```

The stored `rank_tolerance` is squared because eigenvalues of $F^\top F$ are squared singular values of F .

After subtracting a sample mean, the centered configuration matrix has rank at most $(N-1)$. Ordinary batch means has rank at most $(a-1)$, where (a) is the number of complete batches. Thus singularity when $p \geq N$ is often the mathematically correct representation of the available information, not a numerical failure.

5 Bartlett/Newey–West estimator

5.1 Public configuration

The primary estimator is configured explicitly:

```
method = BartlettLongRunCovariance(
    maximum_lag=64,
    stability=BandwidthStabilityCheck(
        comparison_lags=(16, 32),
        relative_tolerance=0.10,
    ),
    wolff_validation=ProjectedWolffValidation(
        exponential_scale=1.5,
        maximum_lag=256,
    ),
)
```

Only `maximum_lag` is required. No estimated diagonal autocorrelation time silently selects the matrix bandwidth.

5.2 Mathematical estimator

For $k \geq 0$, define the implementation’s lag product

$$A_k = \sum_{t=0}^{N-k-1} Z_t Z_{t+k}^\top.$$

With maximum lag m , bandwidth $b = m + 1$, and Bartlett weight

$$w_k = 1 - \frac{k}{b},$$

the uncorrected covariance-of-the-mean estimate is

$$C_{\text{raw}} = \frac{1}{N^2} \left[A_0 + \sum_{k=1}^m w_k (A_k + A_k^\top) \right]. \quad (1)$$

This is the usual Bartlett/Newey–West lag window, expressed directly on the scale of the mean. Newey and West’s Eq. (5) gives the triangular weighting and their theorems establish positive semidefiniteness and consistency under their assumptions: [Newey and West \(1987\)](#).

5.3 Moving sums produce the Bartlett kernel exactly

The triangular kernel is the normalized overlap of two length- (b) boxes. Two boxes displaced by $k < b$ positions overlap at $b - k$ entries, so

$$\frac{b - k}{b} = 1 - \frac{k}{b} = w_k.$$

Zero-pad the centered history by (b-1) rows at each end. For each valid padded position (s), define a rolling vector sum

$$R_s = \sum_{j=0}^{b-1} Z_{s+j},$$

where padded rows contribute zero. Expanding all outer products gives

$$\frac{1}{b} \sum_s R_s R_s^\top = A_0 + \sum_{k=1}^m \left(1 - \frac{k}{b}\right) (A_k + A_k^\top). \quad (2)$$

The implementation computes every R_s through cumulative sums:

```
bandwidth = maximum_lag + 1
padded = np.pad(
    centered,
    ((bandwidth - 1, bandwidth - 1), (0, 0)),
)
cumulative = np.vstack(
    (
        np.zeros((1, value_count)),
        np.cumsum(padded, axis=0),
    )
)
rolling_sums = (
    cumulative[bandwidth:]
    - cumulative[:-bandwidth]
)
```

This is not an approximation to the Bartlett weights. Equation (2) is an algebraic identity.

5.4 Exact correction for sample centering under IID data

Subtracting the sample mean makes distinct centered IID observations weakly anti-correlated. If the original IID covariance is (Ω), then

$$\mathbb{E}[Z_t Z_t^\top] = \frac{N-1}{N} \Omega, \quad \mathbb{E}[Z_t Z_s^\top] = -\frac{1}{N} \Omega \quad (t \neq s).$$

Substituting these expectations into Eq. (1) gives

$$\mathbb{E}[C_{\text{raw}}] = r_{N,m} \frac{\Omega}{N},$$

where

$$r_{N,m} = \frac{1}{N} \left[(N-1) - \frac{2}{N} \sum_{k=1}^m w_k (N-k) \right]. \quad (3)$$

The code calculates Eq. (3) literally:

```
weighted_pair_count = 0.0
for lag in range(1, maximum_lag + 1):
    weight = 1.0 - lag / (maximum_lag + 1.0)
    weighted_pair_count += (
```



```

        weight * (configuration_count - lag)
    )
iid_bias_factor = (
    (configuration_count - 1.0)
    - 2.0 * weighted_pair_count / configuration_count
) / configuration_count

```

The returned estimate divides by $r_{N,m}$. Combining Eqs. (2) and (3), the factor is

$$F = \frac{R}{\sqrt{bN^2 r_{N,m}}}, \quad F^\top F = \frac{C_{\text{raw}}}{r_{N,m}}.$$

The implementation is:

```

factor = rolling_sums / np.sqrt(
    bandwidth
    * configuration_count
    * configuration_count
    * iid_bias_factor
)

```

At $m = 0$, $r_{N,0} = (N - 1)/N$, so dividing by it is exactly Bessel's correction for the covariance of an IID sample mean. The evidence stores `iid_centering_correction = 1.0 / iid_bias_factor`.

This correction is a documented toolkit derivation. It makes the estimator exactly unbiased for IID data after sample centering. It does **not** remove all finite-sample bias for a correlated process, nor does it prove that the chosen lag has captured the tail.

5.5 Structural PSD

The selected covariance is never assembled and clipped. Instead:

```

factor, iid_bias_factor = factor_at(method.maximum_lag)
selected_variances = np.sum(factor * factor, axis=0)
spectrum = factor_spectrum(factor)
values = correlated_values_from_factor(means, factor)

```

Because the estimate is $F^\top F$, the evidence declares no projection:

```

CovarianceProjectionEvidence(
    policy=None,
    projected_mode_count=0,
    minimum_eigenvalue_before=spectrum.minimum_eigenvalue,
    maximum_eigenvalue_before=spectrum.maximum_eigenvalue,
    relative_frobenius_adjustment=0.0,
)

```

5.6 Bandwidth-stability diagnostic

If a `BandwidthStabilityCheck` is present, the code recomputes the complete factor at every declared lower comparison lag. It compares diagonal variances coordinate by coordinate and retains one typed record per comparison:

```

bandwidth_comparisons = []
for comparison_lag in method.stability.comparison_lags:
    comparison_factor, _ = factor_at(comparison_lag)
    comparison_variances = np.sum(
        comparison_factor * comparison_factor,
        axis=0,
    )

```

```

)
scale = np.maximum(
    np.maximum(
        np.abs(selected_variances),
        np.abs(comparison_variances),
    ),
    np.finfo(float).tiny,
)
bandwidth_comparisons.append(
    BandwidthComparisonEvidence(
        comparison_lag=comparison_lag,
        selected_lag=method.maximum_lag,
        maximum_relative_variance_change=float(
            np.max(
                np.abs(selected_variances - comparison_variances)
                / scale
            )
        ),
    )
)
)

worst_comparison = max(
    bandwidth_comparisons,
    key=lambda item: item.maximum_relative_variance_change,
)

```

For coordinate (i), the comparison is

$$d_i = \frac{|C_{ii}(m) - C_{ii}(m_c)|}{\max(|C_{ii}(m)|, |C_{ii}(m_c)|, \text{tiny})}, \quad d_{\max} = \max_i d_i.$$

If $d_{\max}$ exceeds the declared tolerance, the estimate is marked **Unresolved**. By default, this is evidence rather than an exception, so an exploratory analysis still receives a result. A strict analysis can configure

`on_unresolved=UnresolvedAutocorrelationAction.Raise`

and receive a typed **UnresolvedAutocorrelation** failure.

Every declared lower lag is an evaluated plateau point. The worst comparison drives the aggregate status and diagnostic; `bandwidth_comparisons` preserves all individual results and `bandwidth_scan_lags` preserves their order.

5.7 What the stability diagnostic does not prove

It checks the p coordinate variances only. A linear combination $a^T Y$ can expose a slow covariance mode that is weak or invisible on individual diagonals. A stable pair of bandwidths also cannot exclude a correlation tail beyond both bandwidths. Therefore **Resolved** means that the **declared diagnostics** resolved what they tested; it is not a theorem that the entire infinite autocorrelation tail is known.

6 Multivariate non-overlapping batch means

6.1 Construction

Choose batch size (b\$. The implementation uses

$$a = \left\lfloor \frac{N}{b} \right\rfloor$$

complete batches and discards $(N-ab)$ configurations from the end. For batch $j = 0, \dots, a-1$, define

$$\bar{Y}_j(b) = \frac{1}{b} \sum_{t=jb}^{(j+1)b-1} Y_t, \quad \bar{Y}_b = \frac{1}{a} \sum_{j=0}^{a-1} \bar{Y}_j(b).$$

The estimator of the covariance of the overall mean is

$$\hat{C}_{\text{BM}} = \frac{1}{a(a-1)} \sum_{j=0}^{a-1} (\bar{Y}_j - \bar{Y}_b)(\bar{Y}_j - \bar{Y}_b)^\top. \quad (4)$$

This is the batch-means estimator of the long-run covariance divided by the used sample size (ab) . It matches Eq. (10) in [Vats, Flegal, and Jones \(2019\)](#) after putting the result on the covariance-of-the-mean scale.

6.2 Code mapping

The array reshape creates the $a \times b \times p$ batch view:

```
batch_count = configuration_count // method.batch_size
if batch_count < 2:
    raise StatisticsError(
        "batch_size must leave at least two complete batches"
    )
used_configuration_count = batch_count * method.batch_size
discarded_configuration_count = (
    configuration_count - used_configuration_count
)
used_values = values[:used_configuration_count]
batch_means = used_values.reshape(
    batch_count,
    method.batch_size,
    value_count,
).mean(axis=1)
```

The estimator is again expressed as a factor:

```
means = batch_means.mean(axis=0)
centered_batch_means = (
    batch_means - means[np.newaxis, :]
)
factor = centered_batch_means / np.sqrt(
    batch_count * (batch_count - 1.0)
)
```

Thus `factor.T @ factor` is Eq. (4) exactly. It is PSD without projection and has rank at most $(a-1)$.

6.3 A complete scalar worked example

Suppose $N = 8$, $b = 2$, and the history offsets are

$$(-3, -1, -1, 1, 1, 3, 3, 5).$$

The four batch means are

$$(-2, 0, 2, 4),$$

with overall mean (1). The centered batch means are

$$(-3, -1, 1, 3).$$

Their squared sum is (20), so Eq. (4) gives

$$\widehat{\text{Var}}(\bar{Y}) = \frac{20}{4(4-1)} = \frac{5}{3}.$$

This exact example is exercised through `RunningCoupling.process(...)` in `tests/test_long_run_covariance_interface.py`.

6.4 Tail-discard semantics

The returned mean is the mean of `used_values`, not all (N) configurations. The discarded tail therefore affects both the central value and covariance. The evidence records `discarded_configuration_count` so this is never silent. Changing the batch size can change the included sample when $b \nmid N$; a batch-size comparison should account for that fact.

6.5 Statistical limits

Consistency requires both the batch size and the number of batches to grow under the conditions in the cited paper. Two batches satisfy the arithmetic contract, but provide only rank one and very weak finite-sample evidence. Ordinary batch means is often downward biased for strongly positively correlated finite histories. It is an independent PSD comparator, not a guaranteed conservative bound.

7 Positive-leading-bias over-lugsail batch means

7.1 Why this method exists

Lugsail estimators alter a base lag window to control its leading asymptotic bias. Vats and Flegal define

$$k_L(x) = \frac{k(x) - c k(rx)}{1 - c}, \quad r \geq 1, \quad 0 \leq c < 1.$$

The toolkit uses the analogous batch-means combination

$$\widehat{C}_L = \frac{\widehat{C}_b - c \widehat{C}_{b/r}}{1 - c}. \tag{5}$$

For a first-order Bartlett window, $r = 3$, $c = 1/2$ is an over-lugsail choice: $c > 1/r$ reverses the sign of the leading bias term under the paper’s positive-correlation conditions. See Eq. (7), the bias results, and the recommended parameters in [Vats and Flegal \(2022\)](#).

Positive leading bias is not the same as a finite-sample upper confidence bound and is not a Loewner-order guarantee for the true covariance matrix.

7.2 Configuration invariants

The method requires:

$$r \in \mathbb{Z}, \quad r \geq 2, \quad b \bmod r = 0, \quad 0 < c < 1, \quad c > 1/r.$$

The final inequality is what makes this class specifically an **over-lugsail** implementation rather than the full lugsail parameter family. The code rejects incompatible configurations during construction.

7.3 Two ordinary batch estimates on one used sample

First, the method truncates the history to a whole number of large batches. Both component covariances use that same prefix and the same mean. Each is represented by a factor rather than a dense covariance:

```
smaller_batch_size = (
    method.batch_size // method.lugsail_scale
)
batch_count = configuration_count // method.batch_size
used_configuration_count = batch_count * method.batch_size
used_values = values[:used_configuration_count]
means = used_values.mean(axis=0)

def factor_for_batch_size(batch_size: int) -> np.ndarray:
    count = used_configuration_count // batch_size
    batch_means = used_values.reshape(
        count,
        batch_size,
        value_count,
    ).mean(axis=1)
    centered = batch_means - means[np.newaxis, :]
    return centered / np.sqrt(count * (count - 1.0))
```

For the large batch size there are (a) batches; for the small batch size there are (ar) batches. Since the large-batch prefix is divisible by both sizes, the shared `means` is also exactly the mean of either set of batch means.

The lugsail combination is a signed Gram matrix. If `large_factor` is F_b and `small_factor` is $F_{b/r}$, then

$$\widehat{C}_L = R^\top J R,$$

where R stacks the appropriately scaled factors and J has positive signs for large-batch rows and negative signs for small-batch rows:

```
stacked = np.vstack(
    (
        np.sqrt(1.0 / (1.0 - weight)) * large_factor,
        np.sqrt(weight / (1.0 - weight)) * small_factor,
    )
)
signs = np.concatenate(
    (np.ones(large_factor.shape[0]), -np.ones(small_factor.shape[0]))
)
```

7.4 Why projection can be necessary

Both component estimates are PSD, but their weighted **difference** need not be. Constructing a dense $p \times p$ matrix would be prohibitive for the joint high-dimensional case. The implementation instead takes the thin SVD $R = UDV^\top$ and diagonalizes only

$$D(U^\top J U)D,$$

whose dimension is bounded by the number of batch-factor rows, not p :

```
left, singular_values, right = np.linalg.svd(
    stacked, full_matrices=False
)
signed_metric = left.T @ (signs[:, None] * left)
reduced_covariance = (
```

```

    singular_values[:, None]
    * signed_metric
    * singular_values[None, :]
)
eigenvalues, eigenvectors = np.linalg.eigh(reduced_covariance)
rank_tolerance = (
    value_count
    * np.finfo(float).eps
    * float(np.max(np.abs(eigenvalues)))
)
negative = eigenvalues < 0.0
projected_mode_count = int(
    np.count_nonzero(negative)
)

```

The declared nearest-PSD policy clips every negative supported eigenvalue to zero and constructs a projected low-rank factor directly:

```

projected_eigenvalues = np.maximum(eigenvalues, 0.0)
projected_factor = (
    np.sqrt(projected_eigenvalues)[:, None]
    * (eigenvectors.T @ right)
)

```

For a symmetric matrix, this eigenvalue clipping is the nearest PSD matrix in Frobenius norm. The evidence records:

- how many negative supported modes were clipped;
- the minimum and maximum eigenvalues before projection;
- the scale-aware tolerance;
- the relative Frobenius adjustment $\|C_+ - C\|_F / \|C\|_F$.

The decomposition is algebraically equivalent to clipping the eigenvalues of the sample-supported dense covariance; it merely avoids materializing that matrix. The evidence therefore names the implementation `symmetric-covariance-eigenvalue-clipping` and the representation `projected-low-rank-factor`.

This covariance-eigenvalue projection is a declared **toolkit policy**. It is not the positive-definite adjustment in Vats and Flegal’s paper. Their Section 5 adjusts an estimated correlation matrix, floors its eigenvalues at a positive threshold, and then restores marginal variance scales. The toolkit instead clips covariance eigenvalues at zero and can change the diagonal. The paper supports Eq. (5), but not this exact repair.

7.5 Recommended interpretation

Use this as a conservative **systematic variation** for histories dominated by positive persistence. If it exceeds ordinary batch means, that is expected under the motivating bias expansion. If it does not, the data have not violated the implementation; finite-sample behavior and negative correlation can reverse the ordering. A large PSD adjustment is itself evidence that the raw lugsail matrix is poorly determined for the requested dimension and batch policy.

8 Projected scalar Wolff validation

8.1 Its role is validation, not matrix assembly

`ProjectedWolffValidation` applies Wolff’s scalar Gamma-method calculation to each coordinate Y_i and, optionally, to declared linear combinations $a_j^\top Y$. It compares every scalar result with the matching variance induced by the enclosing Bartlett factor.

It does not:

- estimate the full covariance matrix;

- assign a separate automatic window to every matrix entry;
- change the Bartlett covariance;
- inflate the Bartlett covariance when it disagrees;
- silently turn a large resolved variance difference into an inflated error.

Its typed unresolved count participates in Bartlett’s record-or-raise policy. A configured `relative_variance_tolerance` makes a resolved but excessive Bartlett–Wolff disagreement participate in the same policy. Without that optional threshold, the difference remains diagnostic evidence only.

Declared projections must be finite, nonzero, and have exactly the history dimension. No unit-norm rescaling is imposed: the coefficients retain their declared scientific meaning. The identity and declared projections are assembled and applied to both the history and Bartlett factor:

```
assessment_matrix = np.vstack(
    (np.eye(coordinate_count), projection_matrix)
)
centered = centered @ assessment_matrix.T
selected_factor = selected_factor @ assessment_matrix.T
```

8.2 Lag covariance and accumulated variance

For each coordinate, the code uses Wolff’s lag-dependent denominator:

```
gamma_zero = np.sum(centered * centered, axis=0) \
    / configuration_count
covariance_sums = gamma_zero.copy()

lag_covariance = (
    np.sum(centered[:-lag] * centered[lag:], axis=0)
    / (configuration_count - lag)
)
covariance_sums += 2.0 * lag_covariance
```

At candidate window (W), `covariance_sums` is

$$C(W) = \hat{\Gamma}(0) + 2 \sum_{k=1}^W \hat{\Gamma}(k),$$

where

$$\hat{\Gamma}(k) = \frac{1}{N-k} \sum_{t=0}^{N-k-1} Z_t Z_{t+k}.$$

This rectangular scalar sum is Wolff’s object, not the Bartlett matrix estimator.

8.3 Integrated and effective exponential times

The current integrated-time estimate is

$$\hat{\tau}_{\text{int}}(W) = \frac{C(W)}{2\hat{\Gamma}(0)}.$$

The code then solves Wolff’s exponential proxy for an effective time:

```
integrated_times = np.divide(
    covariance_sums,
    2.0 * gamma_zero,
```

```

    out=np.full(value_count, 0.5),
    where=gamma_zero > 0.0,
)
effective_times = np.full(
    value_count,
    np.finfo(float).tiny,
)
positive_time = integrated_times > 0.5
effective_times[positive_time] = (
    validation.exponential_scale
    / np.log(
        (2.0 * integrated_times[positive_time] + 1.0)
        / (2.0 * integrated_times[positive_time] - 1.0)
    )
)

```

Thus

$$\bar{\tau}(W) = \frac{S}{\log[(2\hat{\tau}_{\text{int}}(W) + 1)/(2\hat{\tau}_{\text{int}}(W) - 1)]},$$

with configured $1 \leq S \leq 2$. Coordinates with $\hat{\tau}_{\text{int}} \leq 1/2$ keep a tiny positive effective time; this makes the window function select promptly rather than evaluate an invalid logarithm.

8.4 Automatic window selection

For each still-unselected coordinate, the code evaluates

```

window_function = (
    np.exp(-lag / effective_times)
    - effective_times
    / np.sqrt(lag * configuration_count)
)
newly_selected = (
    (selected_windows < 0)
    & (window_function < 0.0)
)

```

This is Wolff's balance between estimated truncation error and statistical noise:

$$g(W) = e^{-W/\bar{\tau}(W)} - \frac{\bar{\tau}(W)}{\sqrt{WN}}.$$

The first (W) for which (g(W)<0) is selected. At that point the variance of the mean is stored with Wolff's leading finite-(N) correction:

```

wolff_variances[newly_selected] = (
    covariance_sums[newly_selected]
    * (1.0 + (2.0 * lag + 1.0)
        / configuration_count)
    / configuration_count
)

```

That is

$$\widehat{\text{Var}}_W(\bar{Y}_i) = \frac{C_i(W)}{N} \left(1 + \frac{2W + 1}{N}\right).$$

These steps correspond to Wolff's Eqs. (31), (35), and (49)–(52): [Wolff \(2004\)](#).

8.5 Constant and unresolved coordinates

Coordinates with exactly zero centered variance are assigned window zero and variance zero. For all others, the scan stops at

$$\min(\text{validation.maximum_lag}, N - 1).$$

If no sign change occurs before that cap, the coordinate remains unresolved. The evidence stores separate unresolved counts for original coordinates and declared projections, plus the minimum/maximum selected window among resolved assessments. A maximum relative Bartlett–Wolff variance difference is computed only when every coordinate and every declared projection resolves.

This design keeps low-quality data analyzable by default: unresolved evidence travels with the result. It does not pretend that the unseen tail has a known upper bound.

9 Experimental rectangular lag-sum heuristic

9.1 Scientific status

`ExperimentalRectangularLongRunCovariance` is retained for differential comparison with an earlier agent-generated implementation. The transitional alias `GammaMethod` points to it only so early notebooks continue to import; the source explicitly states:

```
# Transitional import alias for early notebooks.
# This is not Wolff's method.
GammaMethod = ExperimentalRectangularLongRunCovariance
```

Its evidence has `source=None`. It must not be cited as Wolff’s Gamma method or treated as a canonical full-covariance extension.

9.2 Diagonal window probe

The heuristic begins with population-normalized centered variances and an integrated time of (1/2):

```
variances = np.sum(centered * centered, axis=0) \
    / configuration_count
safe_variances = np.where(
    variances > 0.0,
    variances,
    1.0,
)
integrated_times = np.full(value_count, 0.5)
maximum_probe = min(configuration_count // 4, 1000)
```

At each probe lag, it forms coordinate autocorrelations

$$\hat{\rho}_i(k) = \frac{\sum_{t=0}^{N-k-1} Z_{ti} Z_{t+k,i}}{N \hat{\Gamma}_i(0)}.$$

Only positive correlations inside a coordinate-dependent provisional window are accumulated:

```
correlations = (
    np.sum(centered[:-lag] * centered[lag:], axis=0)
    / (configuration_count * safe_variances)
)
active = (
    (correlations > 0.0)
    & (lag < method.window_factor * integrated_times)
```

```

)
integrated_times = np.where(
    active,
    integrated_times + correlations,
    integrated_times,
)
if not active.any():
    break

```

This is already materially different from Wolff’s automatic window:

- negative autocorrelations are not accumulated;
- all probing stops as soon as no coordinate is active at one lag;
- the lag normalization is (N), not (N-k);
- there is no Wolff noise-versus-tail window function;
- there is no typed unresolved state.

9.3 One common rectangular matrix window

The largest probed coordinate time chooses one matrix window:

```

window = max(
    4,
    int(np.ceil(
        method.window_factor * float(integrated_times.max())
    )),
)
window = min(window, configuration_count // 4)

```

Then the full symmetric rectangular lag sum is formed:

```

covariance_sum = centered.T @ centered
for lag in range(1, window + 1):
    lagged = centered[:-lag].T @ centered[lag:]
    covariance_sum += lagged + lagged.T
covariance = covariance_sum / (
    configuration_count * configuration_count
)
covariance = 0.5 * (covariance + covariance.T)

```

In formulas,

$$\widehat{C}_{\text{rect}}(W) = \frac{1}{N^2} \left[A_0 + \sum_{k=1}^W (A_k + A_k^T) \right]. \quad (6)$$

Unlike Bartlett’s triangular kernel, a finite rectangular kernel is not guaranteed PSD. The implementation clips every negative eigenvalue to zero and records the relative adjustment. It also applies no IID sample-centering correction to Eq. (6).

9.4 Appropriate use

Use this class only to reproduce or investigate legacy behavior. Its useful properties are that it preserves full cross-covariance, uses one common matrix window, and records its PSD repair. Those properties do not supply a literature justification for its window rule. Bartlett, ordinary batch means, over-lugsail variation, and projected Wolff validation have clearer and separately documented scientific roles.

10 Covariance projection evidence

CovarianceProjectionEvidence separates the numerical output from the intervention used to make it acceptable as a covariance:

```
@dataclass(frozen=True)
class CovarianceProjectionEvidence:
    policy: CovarianceProjection | None
    projected_mode_count: int
    minimum_eigenvalue_before: float
    maximum_eigenvalue_before: float
    relative_frobenius_adjustment: float

    @property
    def applied(self) -> bool:
        return self.projected_mode_count > 0
```

Interpret the fields together:

- policy=None means structural PSD; no repair was part of the method;
- a non-None policy declares that the method permits a repair;
- projected_mode_count=0 means no substantive repair was triggered in that estimate;
- the pre-projection eigenvalue range shows the original matrix scale;
- the relative Frobenius adjustment quantifies how much the reported matrix differs from the raw estimate.

Projection makes a matrix usable as a covariance; it does not make a noisy or under-resolved estimator scientifically accurate. A large adjustment should be treated as a diagnostic, not hidden as routine roundoff.

11 Reading the result report

ProcessingResult.statistical_methods includes the stable estimator identity and source:

```
f"{channel.statistics.estimator} "
f"[source: {channel.statistics.source or 'unavailable'}]"
```

A representative report therefore contains lines of this form:

```
statistics: bartlett-newey-west
  [source: https://doi.org/10.2307/1913610#equation-5]
covariance projections: 0/1 estimates
maximum relative covariance adjustment: 0
unresolved autocorrelation estimates: 1/1
```

The projection count examines one shared statistical estimate per ensemble, not every channel alias. The unresolved line appears only when at least one ensemble carries unresolved evidence.

For programmatic inspection:

```
channel = result.ensembles[0].channels[0]
evidence = channel.statistics

print(evidence.estimator)
print(evidence.source)
print(evidence.configuration_count, evidence.value_count)
print(evidence.numerical_rank, evidence.rank_deficient)
print(evidence.autocorrelation.status)
print(evidence.autocorrelation.diagnostics)
print(evidence.covariance.relative_frobenius_adjustment)
```

12 Method-by-method comparison

Method	Full matrix?	Window/batch choice	PSD behavior	Tail evidence	Intended role
Bartlett/Newey–West	Yes	Explicit maximum lag	Structural factor	Bandwidth check and optional projected Wolff	Primary estimate
Multivariate batch means	Yes	Explicit batch size	Structural factor	Batch-size comparisons are external	Independent comparator
Over-lugsail batch means	Yes	Explicit large/small batches	Difference may be projected	Bias direction recorded, no resolution test	Conservative systematic variation for positive persistence
Projected Wolff	Diagonal only	Automatic scalar window up to explicit cap	Not a matrix estimator	Unresolved coordinate count	Validator for Bartlett diagonals
Experimental rectangular	Yes	Heuristic diagonal probe, one common window	Always eigenvalue-clipped when negative	None	Legacy differential comparison only

13 Failure modes and what is deliberately not promised

13.1 No distribution-free finite-data upper bound

No implemented method guarantees that its covariance is an upper bound on the true covariance for every stationary process. A process can hide covariance at a lag beyond every resolvable lag in a finite history. For example,

$$Y_t = \epsilon_t + \theta \epsilon_{t-L}, \quad \theta > 0,$$

has no covariance at lags $0 < k < L$, but has positive covariance at lag L . If the run cannot resolve (L) , no observed short-lag plateau can discover that contribution without additional model assumptions.

The toolkit therefore distinguishes:

- an estimate, which remains available for exploratory work;
- unresolved evidence, which says the declared diagnostic did not establish a stable tail;
- an over-lugsail systematic variation, which reverses a leading bias under stated positive-correlation assumptions;
- a strict **Raise** policy, which a publication or CI workflow may choose.

13.2 Rank deficiency is not repaired

When $p > N - 1$, the observed centered histories cannot identify all directions in $\mathbb{R}^p$. Bartlett and batch means preserve this null space. They do not add a ridge, manufacture independent noise, or replace the covariance by its diagonal. Any later inverse-covariance fit must declare its own dimension reduction or regularization policy.

13.3 Full covariance does not mean every direction is diagnosed

The primary estimator retains every cross-covariance that its common window includes. Its built-in stability and Wolff checks nevertheless inspect the original coordinate diagonals. A future projection-based diagnostic could also test scientifically chosen linear combinations or empirical slow modes. That is not implemented today.

14 Validation oracles in the test suite

The implementation tests are useful because they exercise the public processing path rather than calling private arithmetic alone.

14.1 Perfect cross-channel correlation

One test constructs a second observable as exactly twice the first on every configuration. After joint estimation and all coupling normalization, the two retained values must have correlation matrix

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

This catches accidental per-channel estimation or covariance destruction.

14.2 Exact IID zero-lag correction

For offsets $(-3, -1, 1, 3)$, the unbiased sample variance is $20/3$, so the variance of the four-sample mean is $5/3$. The zero-lag Bartlett path is required to reproduce it exactly, testing Eq. (3).

14.3 Independent batch-means arithmetic

The eight-point worked example above requires variance $5/3$. It tests batch construction and normalization independently of the production code's matrix operations.

14.4 Known VAR(1) long-run covariance

For

$$X_t = AX_{t-1} + \epsilon_t, \quad \text{Cov}(\epsilon_t) = Q,$$

the exact long-run covariance is

$$\Sigma = (I - A)^{-1}Q(I - A)^{-\top}.$$

A long synthetic two-dimensional history checks that the Bartlett full matrix matches (Sigma/N) , including the off-diagonal term, and that the projected Wolff diagonal comparison is reasonably close.

14.5 Positive-correlation lugsail behavior

A long AR(1) history with coefficient (0.9) supplies a case in which ordinary finite-batch means is downward biased. The $r = 3, c = 1/2$ over-lugsail estimate must exceed ordinary batch means and approach the known asymptotic variance. This is a regression oracle for the motivating regime, not a proof of an upper-bound property.

14.6 Unresolved-but-available and strict rejection

Strongly correlated AR(1) fixtures verify both policies:

- default behavior returns a result with `Unresolved` evidence;
- `UnresolvedAutocorrelationAction.Raise` rejects the same calculation.

This protects the requirement that low-quality data remain analyzable without allowing an unresolved tail to become invisible.

15 Practical configuration patterns

15.1 Exploratory primary analysis

```
statistics = StatisticsConfiguration(  
    BartlettLongRunCovariance(  
        maximum_lag=64,  
        stability=BandwidthStabilityCheck(  
            comparison_lags=(16, 32),  
            relative_tolerance=0.10,  
        ),  
        wolff_validation=ProjectedWolffValidation(  
            exponential_scale=1.5,  
            maximum_lag=256,  
        ),  
    ),  
)
```

This returns an estimate even if a diagnostic is unresolved, and records why.

15.2 Strict publication gate

```
statistics = StatisticsConfiguration(  
    BartlettLongRunCovariance(  
        maximum_lag=64,  
        stability=BandwidthStabilityCheck(  
            comparison_lags=(16, 32),  
            relative_tolerance=0.10,  
        ),  
        on_unresolved=(  
            UnresolvedAutocorrelationAction.Raise  
        ),  
    ),  
)
```

15.3 Independent batch-size sweep

Run separate immutable processing configurations, for example with batch sizes (16,32,64), and compare scientifically relevant variances and projections. Remember that different sizes may discard different tail counts.

```
StatisticsConfiguration(  
    MultivariateBatchMeans(batch_size=32)  
)
```

15.4 Positive-leading-bias variation

```
StatisticsConfiguration(  
    LugsailBatchMeans(  
        maximum_lag=64,  
        stability=BandwidthStabilityCheck(  
            comparison_lags=(16, 32),  
            relative_tolerance=0.10,  
        ),  
        wolff_validation=ProjectedWolffValidation(  
            exponential_scale=1.5,  
            maximum_lag=256,  
        ),  
    ),  
)
```

```

        batch_size=60,
        lugsail_scale=3,
        lugsail_weight=0.5,
    )
)

```

Always inspect `covariance.projected_mode_count` and `relative_frobenius_adjustment` alongside the resulting uncertainty.

16 Computational characteristics

Let (N) be the configuration count, (p) the number of joint values, (m) the Bartlett maximum lag, and (a) the batch count.

- Joint history storage is $O(Np)$.
- Bartlett rolling sums are computed by cumulative sums rather than a Python loop over all matrix lag products. Its factor has $(N+m)$ rows, though its centered rank remains at most $(N-1)$.
- Factor rank evidence requires an SVD. The latent path avoids constructing a dense $p \times p$ covariance when p exceeds the factor row count.
- Batch means reduces the history to an $a \times p$ factor before rank and correlation construction.
- Lugsail works in the row space of its stacked batch factors. A thin SVD and reduced eigendecomposition cost approximately $O(q^{2p+q3})$, where (q) is the total number of large- and small-batch rows; it does not construct a dense $(p \times p)$ covariance.
- Experimental rectangular estimation likewise constructs and diagonalizes a dense $p \times p$ matrix.
- Projected Wolff validation scans scalar coordinate and declared-projection products through its lag cap; it avoids $(p \times p)$ lag matrices.

These implementation choices affect scale, not scientific meaning. In particular, a low-rank latent `gvar` representation retains exact correlations present in the factor.

17 Source ledger

Implemented element	Primary source	Implementation status
Full covariance target and scalar Gamma method	U. Wolff, <i>Monte Carlo errors with less errors</i> , DOI 10.1016/S0010-4655(03)00467-3	Projected scalar validator only
Bartlett triangular lag window and PSD HAC construction	W. K. Newey and K. D. West, DOI 10.2307/1913610	Primary full-matrix estimator
Multivariate spectral-variance consistency	D. Vats, J. M. Flegal, and G. Jones, arXiv:1507.08266	Theoretical context for common-window full matrices
Multivariate batch means	D. Vats, J. M. Flegal, and G. Jones, DOI 10.1093/biomet/asz002	Independent full-matrix comparator
Lugsail windows and over-lugsail bias direction	D. Vats and J. M. Flegal, DOI 10.1093/biomet/asab049	Positive-leading-bias systematic variation
Rectangular diagonal-probe heuristic	No canonical source	Explicitly experimental legacy comparator
Exact IID Bartlett sample-centering factor, Eq. (3) above	Direct toolkit derivation	Recorded separately; not attributed to Newey–West

18 File map

Concern	Implementation region
Protocol, validation, evidence, factor propagation, rank	<code>statistics/core.py</code>
Bartlett/Newey–West and stability policy	<code>statistics/bartlett.py</code>
Ordinary multivariate batch means	<code>statistics/batch_means.py</code>
Positive-leading-bias over-lugsail variation	<code>statistics/lugsail_batch_means.py</code>
Projected scalar Wolff validator	<code>statistics/wolff.py</code>
Quarantined rectangular heuristic	<code>statistics/experimental_rectangular.py</code>
Aggregate statistical imports	<code>statistics/__init__.py</code>
Joint history assembly and result reporting	<code>stages/process.py</code>
Higher-level method injection	<code>setup.py</code>
Public processing oracles	<code>test_long_run_covariance_interface.py</code>

All `statistics/` paths in this table are relative to `src/gfrgtoolkit/stages/`; `stages/process.py` and `setup.py` are relative to `src/gfrgtoolkit/`; the test file is relative to `tests/`.